

Saliency-Aware Neural Network Training: Workflow Summary

This document describes the computational workflow for benchmarking saliency-aware CNN training, implemented across two experimental settings: synthetic DNA sequences (`toy_single_arch.py`) and real genomic phenotype prediction (`optuna_phenotype.py`).

Part 1: Common Workflow Concepts

1.1 Two-Phase Hyperparameter Optimization

Both experiments follow a two-phase strategy:

Phase	Goal	What is tuned	What is fixed
Phase 1 (Standard)	Find best CNN architecture	Architecture + optimizer	Training regime (clean inputs)
Phase 2 (Robust)	Find best robustness hyperparameters	Regime type, epsilon/flip_fraction, scheduling	Architecture from Phase 1

Rationale: Decoupling architecture search from robustness tuning reduces the search space and ensures fair comparison of training regimes.

1.2 CNN Architecture (Shared Design)

Both experiments use a 1D CNN with the following pattern:

```
[Conv1 (large kernel) + BN + Activation + Dropout + Pool] ->
[Conv2..N (small kernels) + BN + ReLU + Dropout + Pool]* ->
[Global Pool -> FC -> Logits]
```

Key interpretability-favoring choices: - Large first-layer kernel (`k1`) to capture motif-scale patterns - Exponential activation (`exp`) option in first layer for sharper feature detection - Aggressive early pooling to reduce sequence length while preserving positional information

1.3 Training Regimes

Regime	Description	Key Parameter
Standard	Clean inputs, BCE/CE loss	-
Random Smoothing	Replace one-hot with Dirichlet samples	epsilon in [0.001, 0.5]
Gaussian Smoothing	Add $N(0, \sigma^2)$ noise to inputs	sigma2 in [1e-5, 0.2]
HotFlip (Iterative)	Gradient-based single-flip per iteration	max_flip_fraction in [0.001, 0.3]
Direct HotFlip	Single gradient, apply top-k flips at once	max_flip_fraction in [0.001, 0.5]

Flip Scheduling (HotFlip variants): - **linear**: Ramp from 1 flip to **max_flips** linearly over epochs - **cosine**: Smooth cosine ramp (slower start/end) - **none**: Apply full **max_flips** from epoch 1

1.4 Optimization Setup

Component	Configuration
Optimizer	AdamW with tunable LR and weight decay
Scheduler	ReduceLROnPlateau (factor=0.5, patience varies)
Mixed Precision	AMP with GradScaler
Gradient Clipping	Tunable threshold (typically 0.5-10.0)
Early Stopping	Patience-based on validation loss

1.5 Optuna HPO Configuration

Setting	Value
Sampler	TPE (Tree-structured Parzen Estimator)

Setting	Value
Pruner	Hyperband (min_resource=5, reduction_factor=3)
Objective	Weighted combination of saliency metric + accuracy
Parallelization	Multiple SLURM workers share study via MySQL/PostgreSQL

1.6 Saliency Evaluation

Method: Integrated Gradients (IG) with gradient correction

Common Metrics:

Metric	Definition
Saliency AUC	$P(\text{attribution_inside} > \text{attribution_outside})$ for ground truth regions
Saliency SNR	$\text{Sum}(\text{attr}^2_{\text{inside}}) / \text{Sum}(\text{attr}^2_{\text{total}})$

Part 2: Synthetic Dataset Experiment

Script: toy_single_arch.py

2.1 Synthetic Data Generation

Parameter	Value
Sequence length	1000 bp
Motif length	60 bp
Dataset size	10,000 sequences (balanced)
Positive class	Background + planted motif at random position
Negative class	Pure background (no motif)

Dataset Grid:

Parameter	Values	Count
GC_HPARAMS	0.50, 0.53, 0.55, 0.575, 0.60, 0.625, 0.65, 0.675, 0.70, 0.725, 0.75, 0.775, 0.80	13
CONS_HPARAMS	0.60, 0.65, 0.70, 0.75, 0.80	5
Total	13 x 5 = 65 datasets	

Confounding Design: Negative class always has GC = 0.50, creating a spurious GC-content shortcut.

2.2 Architecture Search Space

Parameter	Range	Scale
k1	7-41 (odd)	linear
k2, k3	3-11 (odd)	linear
c1	16-128	log
c2	32-256	log
c3	64-384	log
pool_w	1-100	log
act1	{exp, relu, gelu, softplus}	categorical
drop_conv1/2/3	0.0-0.4	linear
drop_fc	0.2-0.7	linear

Optimizer Search Space:

Parameter	Range	Scale
lr	1e-4 - 3e-3	log
weight_decay	1e-8 - 1e-2	log
train_batch_size	256-1024	step=128
grad_clip	0.5-10.0	linear

2.3 Evaluation Metrics

Metric	Description
Saliency AUC	Primary objective (see Section 1.6)
wIoU	IoU between top-60bp attribution window and ground truth motif
Saliency SNR	Signal-to-noise ratio of attributions
PGD Success Rate	Fraction of samples where PGD finds adversarial baseline

Effect Size Monitoring (during training): - **GC effect**: $|\Delta \text{logit}|$ when neutralizing background GC - **Motif effect**: $|\Delta \text{logit}|$ when ablating motif region - **Effect ratio**: $\text{motif_effect} / \text{gc_effect}$ (higher = less reliance on confounder)

2.4 Phase 1: Standard HPO

```
sbatch --array=1-N slurm_scripts/run_singlearch_standard.sh
```

Objective: $0.9 \times \text{SaliencyAUC} + 0.1 \times \text{ValAccuracy}$ (averaged over dataset panel)

Dataset Evaluation Modes: - `--eval-mode panel`: 21 representative datasets (7 GC x 3 conservation) - `--eval-mode all`: All 65 datasets - `--eval-mode progressive`: Growing sample with trial number

2.5 Phase 2: Robust HPO

```
sbatch --array=0-64 slurm_scripts/run_singlearch_robust.sh
```

Per-dataset studies: Array index maps to (GC, conservation) pair

Fixed from Phase 1: Architecture (`--robust-arch-from-summary`)

Tuned: - **regime**: {random_smoothing, gaussian_smoothing, hotflip, direct_hotflip} - **schedule**: {on, off} (for HotFlip variants) - Regime-specific epsilon/flip_fraction

2.6 CLI Reference (Synthetic)

Argument	Description
--tune	Run Optuna HPO
--mode {standard,robust}	HPO phase
--study-name NAME	Optuna study identifier
--n-trials N	Trials per worker
--max-epochs N	Max epochs per trial
--eval-mode {panel,all,progressive}	Dataset coverage
--robust-arch-from-summary PATH	Fix architecture from standard summary
--dataset-gc VALUE	GC for per-dataset robust study
--dataset-cons VALUE	Conservation for per-dataset robust study

Part 3: Phenotype (Real Genomic Data) Experiment

Script: `phenotype/code/optuna_phenotype.py`

3.1 Data Characteristics

Property	Value
Input	Whole-genome FASTA files
Sequence length	1,000,000 bp windows (randomly sampled)
Labels	Binary phenotype from metadata Excel
Default phenotype	“Spore formation”
Train/Val/Test	Pre-split directories

Data Loading: - Sequences cached in memory across Optuna trials - Random window sampling during training - Deterministic sampling for validation (reproducibility)

3.2 Architecture Search Space

Parameter	Range	Scale	Notes
k1	51-401 (odd)	linear	Much larger for genomic scale
c1	32-128 (or 64-256 extended)	log	

Parameter	Range	Scale	Notes
stride1	5-50	log	Aggressive downsampling
pool1_k	10-128	log	
pool1_s	5-50	log	
n_blocks	1-3	categorical	
k_small	3-13 (odd)	linear	Depth of small-kernel blocks
c2	64-192 (or 128-384)	log	
c3	128-384 (or 256-768)	log	
fc_hidden	128-512 (or 256-1024)	log	
act1	{exp}	fixed	Kept at exp for interpretability
global_pool	{avg, max}	categorical	

Extended Capacity Mode (--extended-capacity): - Wider channel ranges
- Prefer deeper stacks (2-3 blocks) - Larger FC hidden dimensions

3.3 Evaluation Metrics

Metric	Description
Balanced Accuracy	Primary objective (handles class imbalance)
Saliency AUC	IG-based attribution quality
SaSNR	Saliency signal-to-noise ratio
SaSNR_expected	Expected SaSNR under null (random baseline)
Wilcoxon p-value	Statistical significance of SaSNR > expected
Delta Median CI	Confidence interval for (SaSNR - expected) difference

Objective for Robust Phase: $0.9 \times \text{SaliencyAUC} + 0.1 \times \text{Accuracy}$

3.4 Phase 1: Standard HPO

`sbatch slurm_scripts/run_optuna_phenotypes_std.sh`

Key Differences from Synthetic: - Single phenotype dataset (not a grid)
- Balanced accuracy as objective (not weighted saliency+accuracy) - Class-weighted CrossEntropyLoss for imbalanced labels - Best model saved per phenotype under `phenotype/bacillales_model/<phenotype_slug>/`

3.5 Phase 2: Robust HPO

`sbatch slurm_scripts/run_optuna_phenotypes_robust.sh`

Enabled via: `--robust-epsilon-only`

Fixed from Phase 1: - Architecture loaded from `SporulationModel(summary_path=...)`
- Optimizer settings (LR, weight decay, batch size)

Tuned: - `max_flip_fraction` in `[1e-6, 0.1]` (log scale, via `--eps-min`, `--eps-max`)

Scheduling: Cosine ramp of flip count over epochs (configurable via `--robust-schedule`)

Evaluation Pipeline: - Calls external `phenotype/code/evaluation.py` sub-process - Computes full saliency metrics on test set - Saves best model to `phenotype/robust_model/<study_name>/`

3.6 CLI Reference (Phenotype)

Argument	Description
<code>--tune</code>	Run Optuna HPO
<code>--study-name NAME</code>	Study identifier
<code>--phenotype-col COL</code>	Metadata column for labels (default: "Spore formation")
<code>--metadata-xlsx PATH</code>	Excel file with phenotype annotations
<code>--seq-len N</code>	Window length (default: 1,000,000)
<code>--epoch-budget N</code>	Training samples per epoch
<code>--extended-capacity</code>	Use wider architecture search ranges
<code>--robust-epsilon-only</code>	Enable robust-phase epsilon tuning
<code>--eps-min, --eps-max</code>	Flip fraction search bounds
<code>--robust-schedule</code>	Flip scheduling strategy
<code>{cosine,linear,none}</code>	
<code>--summary-path PATH</code>	Standard-phase summary for architecture loading

Summary Comparison

Aspect	Synthetic	Phenotype
Data	Generated (1000 bp, planted motif)	Real genomes (1M bp windows)
Dataset Grid	65 (GC x conservation)	Single phenotype
Confounder	GC content shortcut	Unknown biological confounders
Primary Metric	Saliency AUC	Balanced Accuracy
Ground Truth	Exact motif positions	Approximate (promoter regions)
Phase 2 Scope	Tune regime + epsilon per dataset	Tune epsilon only (fixed regime)
Evaluation	In-script	External subprocess